PORTFOLIO

한정현

Han Jung-hyun

Python·SQL로 분석하고, scikit-learn·PyTorch로 모델을, FastAPI로 서비스까지 만드는 데이터/AI 엔지니어

Email twinking3594@gmail.com **GitHub** https://github.com/han00jh

kakao han9458

Notion https://diligent-red-1df.notion.site/Han-JeongHyun-s-Portfolio-bf9f8e2e3f2a826b9a7981d8d97577b1?source=copy_link



한 정 현

Email : twinking3594@gmail.com

Git : https://github.com/han00jh

Kakao : Han9458

데이터를 의심하고 확인해, 믿을 수 있는 결과로 만드는 개발자

안녕하십니까. 데이터를 인사이트로, 인사이트를 '돌아가는 제품'으로 이어온 데이터·AI 개발자 한정현입니다.
데이터 분석과 EDA부터 모델링, 서비스 구현, 배포에 이르는 전 과정을 경험하며, 분석 자동화와 사용자 중심의 데이터 서비스를 통해 실질적인 결과를 만들어 왔습니다.
특히 '되는 것처럼 보이는 것'과 '정말 되는 것'을 구분하는 데 강점이 있습니다. 통신 고객 이탈 예측 프로젝트에서는 데이터 누수를 직접 진단·검증해 ROC-AUC 0.906, 실제 이탈자 77% 사전 포착이라는 신뢰할 수 있는 결과를 확보했고, 28개 AI 에이전트 기반 AutoAI 분석 플랫폼(ADA)에서는 정형 분석 파이프라인과 발표자료 자동 생성을 맡아 사람이 며칠 걸리던 분석을 자동화했습니다.
또한 단순히 분석에 그치지 않고, 서비스 구조를 이해하기 위해 FastAPI·PostgreSQL·Docker·CI 등 백엔드와 인프라 구축에도 직접 참여하며, 분석 결과를 실제 제품으로 구현하고 개발자와 원활히 협업하는 기반을 다졌습니다.
앞으로도 화려한 결과보다 믿을 수 있는 결과로 사용자에게 실질적 가치를 전하기 위해, 데이터에서 제품까지 직접 확인하며 만드는 개발자가 되겠습니다.

#데이터·AI개발자 **#데이터로 증명** **#의심하고 검증하는**

About Me

EDUCATION

평택대학교 정보통신학과 [2019.03~2023.02]

EDUCATION

삼성 AXI · AI/데이터 분석 실무 과정 [2025.12~2026.06]

CERTIFICATE & AWARD

OPIc – IM1

JLPT – JLPT N3

TECHNICAL SKILLS

데이터 분석

Python · pandas · NumPy · SciPy · SQL

머신러닝 · 딥러닝

scikit-learn · XGBoost · LightGBM · CatBoost · PyTorch · FLAML · SHAP

AI · LLM

LangChain · LangGraph · LangSmith · RAG · Claude · Ollama

백엔드

FastAPI · Flask · WebSocket · 비동기(asyncio)

DB · 인프라

PostgreSQL(pgvector) · Oracle · MLflow · Docker · Celery

시각화 · 도구

Matplotlib · Seaborn · Chart.js · Git · VSCode · PyCharm · Jupyter

Projects

3개 팀 프로젝트 · 분석 → 모델 → 서비스

01

2026.05–06

ADA

AutoAI · Tabular ML

핵심 문제

비전문가의 데이터 분석 병목

해결 아키텍처

28개 AI 에이전트 오케스트레이션 · 5단계 자가복구

주요 성과

분석 8단계 자동화 · PPT 무편집 생성

02

2026.04

My Mini Co-di

Full-stack · AI 서비스

핵심 문제

옷장 관리와 날씨·TPO 추천의 파편화

해결 아키텍처

FastAPI 비동기 서버 · 실시간 음성(Barge-in) · 성장 루프

주요 성과

CPU 10~60초 재학습 · 무중단 배포

03

2026.02

고객 이탈 예측

Data Analysis · ML

핵심 문제

11% 불균형 데이터의 이탈자 사전 식별

해결 아키텍처

통계적 가설 검증 · LightGBM 실시간 서빙 대시보드

주요 성과

ROC-AUC 0.906 · Recall 77% · Brier 0.060

01 ADA

AutoAI · Tabular ML

대화형 AutoAI 분석 스튜디오 (28 에이전트)

기간 2026.05~06 **팀** 4인 팀 **역할** 정형(Tabular ML/DL) 분석 종주 + PPT 산출물 자동생성

기술 Python · scikit-learn · XGBoost/LightGBM · TabTransformer · FLAML · MLflow · FastAPI · Docker

https://github.com/han00jh/ADA.git

개요 · 목표

정형 데이터를 올리면 28개 AI 에이전트가 자동 분석·튜닝·해석하고
5종 산출물(PPT·PDF·대본·웹·MD)까지 생성하는 대화형 AutoAI 스튜디오

주요 기능 · 구성

- **28개 전문 에이전트 · 5단계 HITL 게이트 + PII 게이트**
- **4개 분석 카테고리: 정형 ML·DL · 시계열 · 이상탐지**
- **3-Stack 자가학습 KB + Guardian 5단계 자동 오류수정(스스로 복구)**
- **보안 풀스택(JWT·OAuth·RBAC·Vault) · 운영 콘솔**



ADA 메인화면

핵심 성과

정형 E2E 3종 통과 · ML/DL 모델 12종 자동 선택 · **19슬라이드 PPT 무편집 자동 생성**

하네스 엔지니어링 × 자가학습 KB

실행 인프라 위에서 자가학습 KB가 돌며, 시스템이 스스로 오류를 고친다

① 하네스 — 5-Tier 자가치유 실행 인프라

Tier 0 · Static Fixers
LLM 호출 없이 결정론적 패턴으로 100ms 내 즉시 복구

▼

Tier 1 · Error KB
과거 실패 교훈을 pgvector로 검색, 신뢰도 ≥ 0.7 패치 즉시 적용

▼

Tier 2 · Ollama (qwen2.5)
로컬 LLM으로 5~15초 내 코드 자동 수정

▼

Tier 3 · Claude CLI
최후 폴백 — 클라우드 LLM으로 10~30초 딥 리커버리

② 자가학습 KB — pgvector KBRAG

= 하네스 Tier 1(Error KB)을 구동하는 동일 엔진

`ada/harness/rag.py`

```
async def search_lessons(self, query, k=5):
    emb = self.embed(query)
    rows = await db.execute(text("""
      SELECT kb.payload, kb.confidence,
        1-(le.embedding <=> :emb) AS sim
      FROM lesson_embeddings le
      JOIN self_learning_kb kb ON ...
      WHERE kb.kb_type = 'failure_lesson'
      ORDER BY sim DESC LIMIT :k """))
    return [r for r in rows if r.sim >= 0.7]
```

Error 발생 ▶ Vector DB 검색 ▶ 유사도 ≥ 0.7 ▶ 자동 수정·실행

API 비용 절감
검증된 로컬 패치 우선 적용으로 클라우드 호출 최소화

자율 복구
코드 실패·Hang까지 사람 개입 없이 스스로 복구

신뢰성 ↑
쓸수록 KB가 축적되어 복구 정확도가 누적 개선

트러블슈팅 — ADA

문제 상황을 가정한 설계로 신뢰성을 확보했습니다.

01

문제 LLM 출력이 실행마다 달라져 슬라이드 구성이 불안정
해결 13개 슬라이드 ID 고정으로 라우팅 결정화 (determinism)
효과 → **무편집 PPT 자동 생성 안정화**

02

문제 ID·PK성 컬럼이 학습에 포함돼 데이터 누수 발생
해결 id-like 컬럼 자동 탐지·제외
효과 → **비현실적 고성능 차단 → 신뢰 가능한 지표**

03

문제 단일 슬라이드 오류가 덱 전체 실패로 번짐
해결 슬라이드별 try/except 격리
효과 → **부분 실패 허용 → 산출물 항상 생성**

28개 에이전트

슈퍼바이저 1

합의 게이트 6

산출물 5종

재학습 루프



뉴럴 오케스트레이션 코어

02

My Mini Co-di

Full-stack · AI 서비스

AI 스타일리스트 웹 서비스

기간 2026.04 **팀** 4인 팀 **역할** Flask→FastAPI 전환 · 날씨 추천 엔진 · AI 통합 · Cloudinary

기술 FastAPI · PostgreSQL(pgvector) · OpenCLIP · Google OAuth · Cloudinary · Three.js · Docker

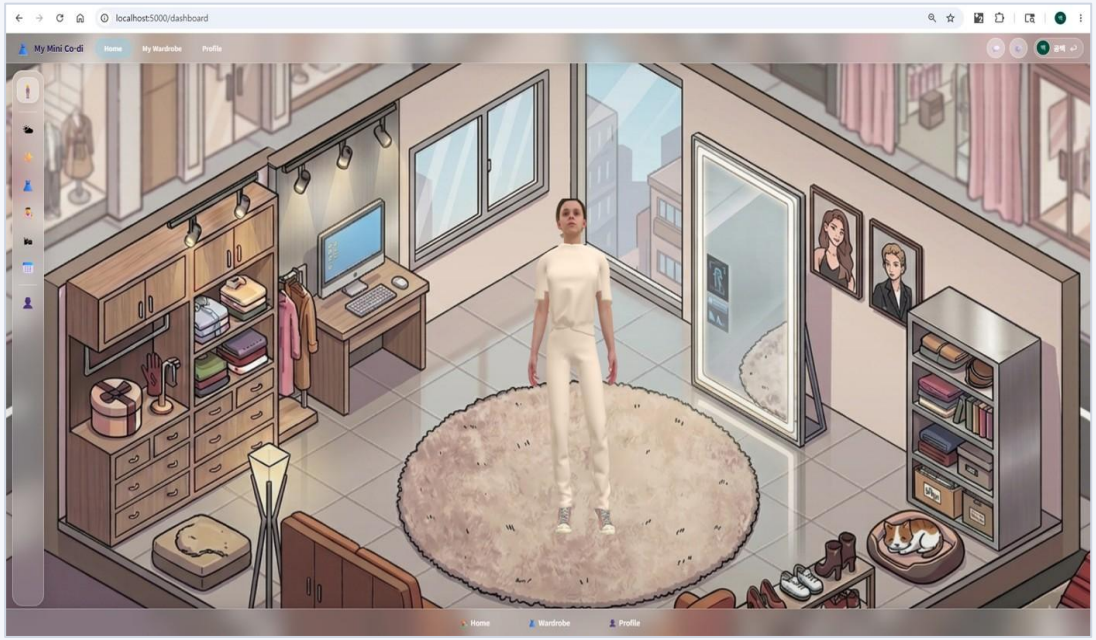
https://github.com/han00jh/CW.git

개요 · 목표

옷장 사진을 올리면 날씨·일정에 맞는 코디를
AI 수석 디자이너가 음성으로 안내하는 풀스택 AI 스타일리스트 웹 서비스

주요 기능 · 구성

- AI 옷장 관리: CLIP 분류(6카테고리·59라벨) · 정정→재학습
- 날씨 기반 추천(기상청) · 구글 캘린더 TPO · 무신사 쇼핑
- AI 수석 디자이너 코멘트(Claude) — 실제 옷 이름 언급
- 실시간 음성 대화: STT + WebSocket + TTS, barge-in



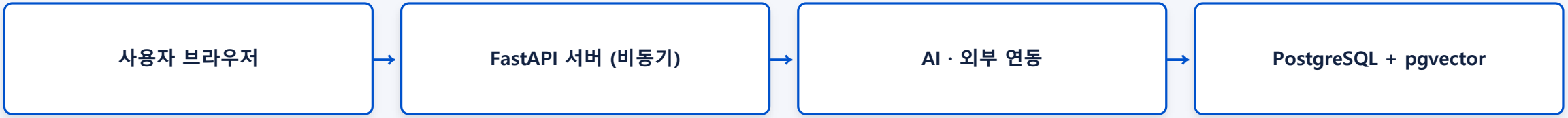
3D 캐릭터 대시보드

핵심 성과

옷장 분류 ~ 날씨 추천 ~ 실시간 음성 코칭까지 동작하는 v1.0 풀스택 웹

시스템 아키텍처

FastAPI 백엔드가 AI 분석·외부 API·이미지 인프라를 비동기로 통합



AI 분석

OpenCLIP(FashionSigLIP) 옷 분류 + 배치 추론

날씨 엔진

기상청 API + 캐시·폴백, 스타일 매핑

Calendar

Google OAuth + 일정 기반 TPO

Cloudinary

사용자별 폴더 + 업로드 정합성

핵심 기여 Flask→FastAPI 전면 전환 · asyncio.to_thread로 CPU-bound 논블로킹 · 예보 회차 캐시+폴백 · 업로드 실패 시 DB 저장 차단(정합성)

트러블슈팅 — My Mini Co-di

문제 상황을 가정한 설계로 신뢰성을 확보했습니다.

01

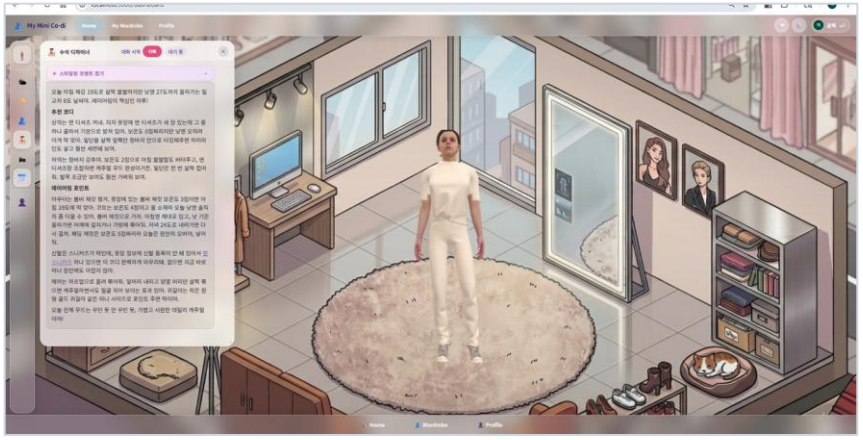
문제 외부 날씨 API 반복 호출로 비용·지연 발생
해결 예보 회차 메모리 캐시 + 캐시→기본값 폴백
효과 → **무중단 운영 · 응답 지연 감소**

02

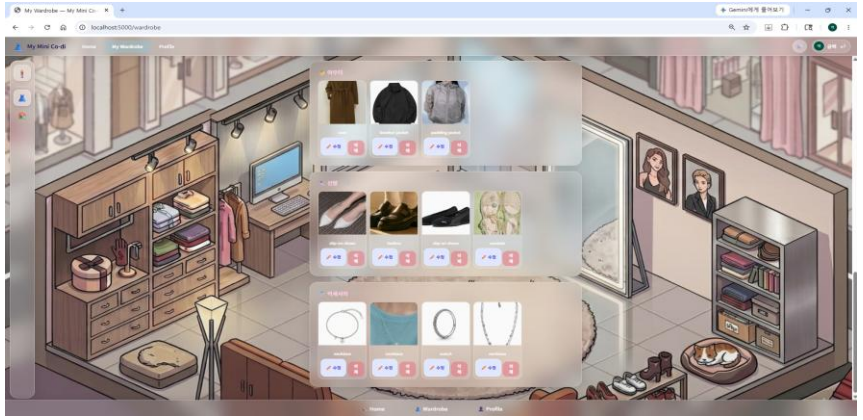
문제 CPU-bound AI·API 작업이 서버를 블로킹
해결 asyncio.to_thread()로 논블로킹 처리
효과 → **동시 요청 처리 · UX 개선**

03

문제 이미지 업로드 실패 시 깨진 데이터가 DB에 저장
해결 업로드 실패 시 RuntimeError로 DB 저장 차단
효과 → **데이터 정합성 보장**



AI 수석 디자이너 코멘트



AI 의류 자동 분류

03 고객 이탈 예측

Data Analysis · ML

통신 고객 이탈 예측 & 유지 전략 대시보드

기간 2026.02 **팀** 4인 팀 **역할** EDA · 파생 피처 엔지니어링 · 전처리 파이프라인 · Oracle DB

기술 Python · scikit-learn · LightGBM · pandas · Flask · Oracle DB · Chart.js · Groq/Claude/Gemini

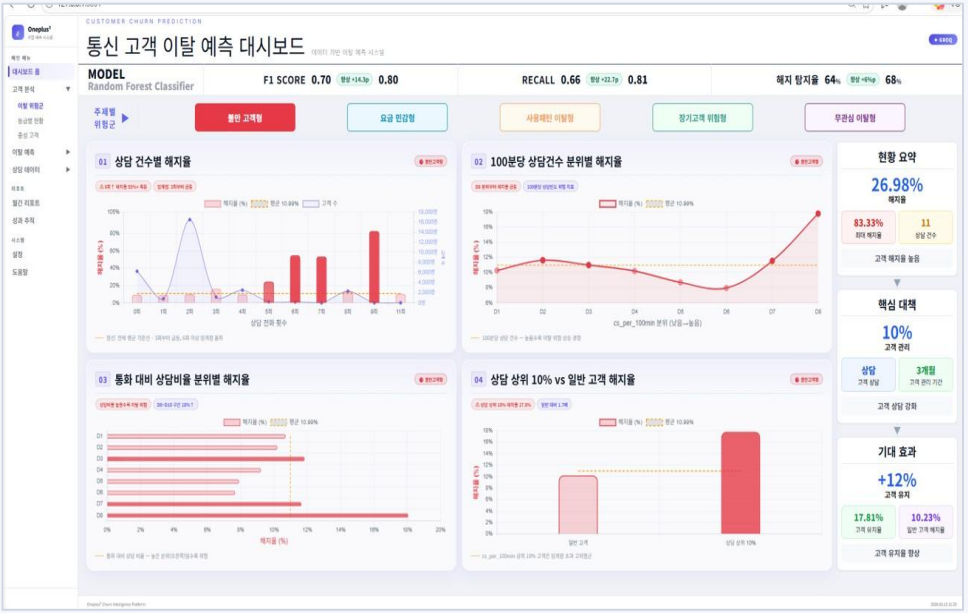
https://github.com/han00jh/churn_semi_porject.git

개요 · 목표

가설검증→통계→다중모델→실시간 대시보드로 통신 고객 이탈을 예측하고 유지 전략을 제시하는 End-to-End 시스템 (30,198명·이탈 11%)

주요 기능 · 구성

- **가설 EDA + 통계검정(Cohen's d·Mann–Whitney·VIF·Kaplan–Meier)**
- **행동 기반 파생피처 17개 · 전처리(RobustScaler·이상치 흡수)**
- **다중 모델링: LogReg·RF·XGBoost·LightGBM (교차검증)**
- **실시간 예측 대시보드(Chart.js) + LLM 인사이트(Groq·Claude·Gemini)**



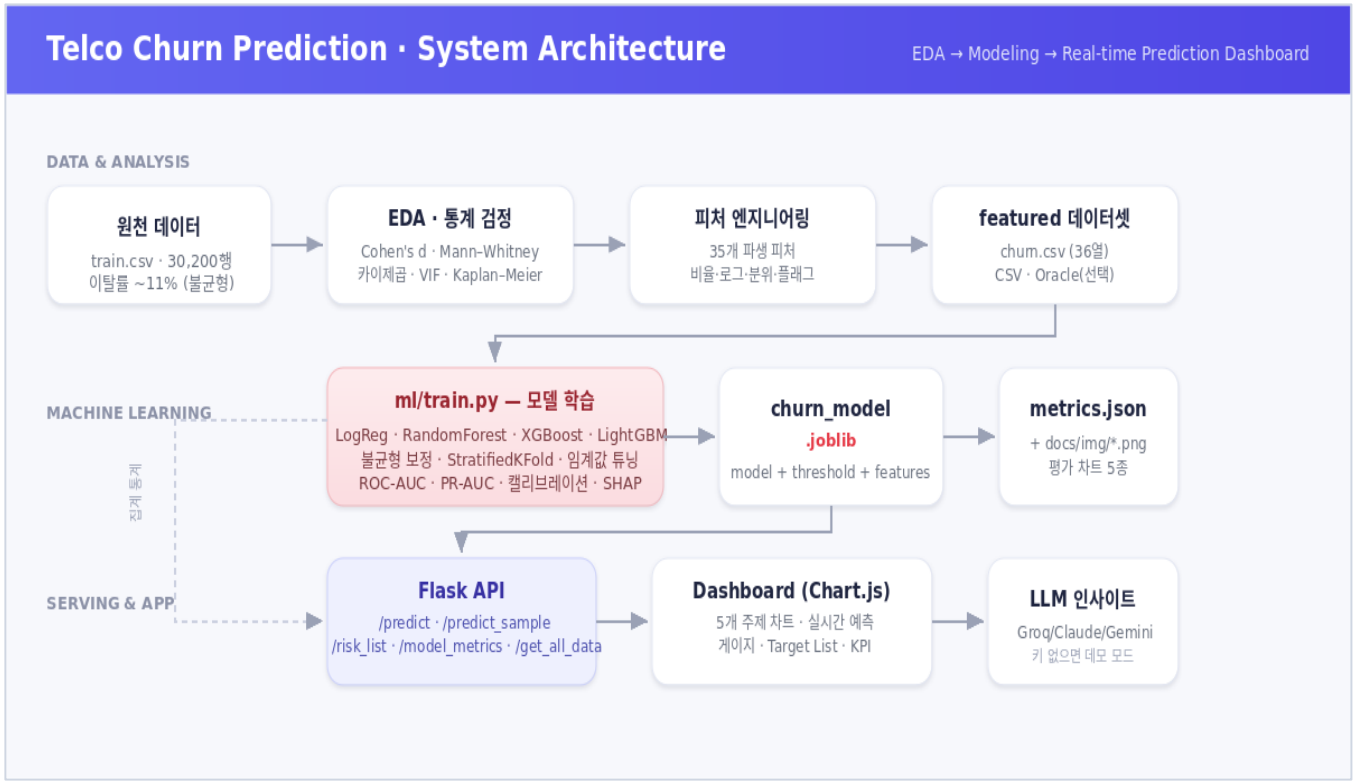
서비스 대시보드 · 운영 RF (F1 0.80)

핵심 성과

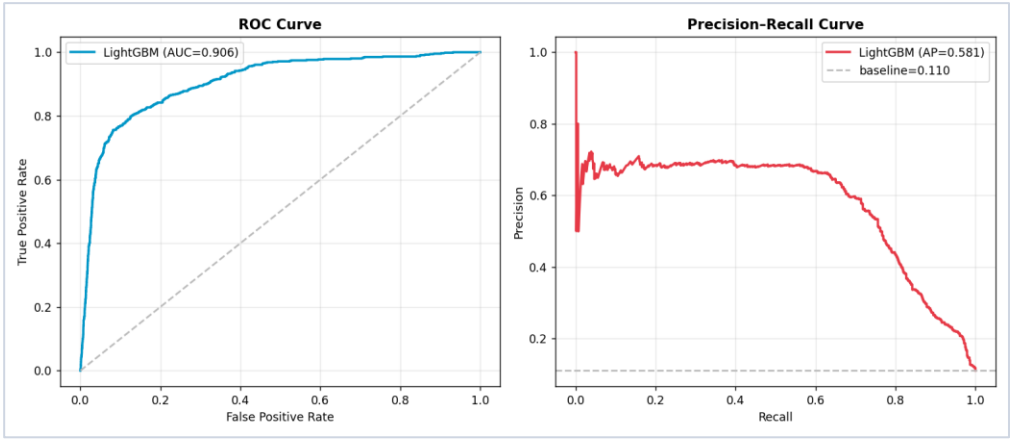
LightGBM ROC-AUC 0.906 · Recall 77% · PR-AUC 0.581 · 통념 뒤집어 '장기·고사용 최고위험' 통계 입증

시스템 아키텍처 & 모델 성능

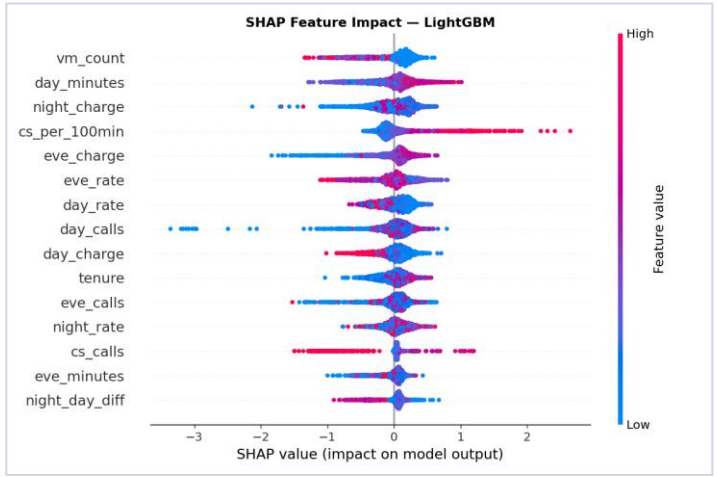
EDA→피처→모델→서빙 전 과정과 평가 지표(ROC·SHAP)



Telco Churn — System Architecture



ROC · PR (AUC 0.906 · AP 0.581)



SHAP 변수 기여 — 상담·요금·시간대

가설 검증 & 파생 피처 설계

비즈니스 가설을 통계로 검증하고, 행동 패턴을 17개 피처로 수치화

6대 비즈니스 가설 (통계 검증)

상담 전화 많을수록 이탈↑	지지
가입기간 짧을수록 이탈	기각
저사용 고객 이탈	반전
야간 통화 집중과 연관	지지
상담 비율 높을수록 이탈	지지
시간당 요금 높을수록 이탈	지지

통념과 달리 '장기·고사용 고객'이 최고위험군임을 입증

행동 기반 파생 피처 17개

```
EPS = 1e-9
df["total_calls"] = (df["day_calls"]
    + df["eve_calls"] + df["night_calls"])
tot = df["total_minutes"] + EPS
# 시간대별 사용 비율
df["day_ratio"]   = df["day_minutes"]   / tot
df["night_ratio"] = df["night_minutes"] / tot
# 밤-낮 편중 · 시간대 집중도
df["night_day_diff"] = df["night_ratio"] -
df["day_ratio"]
ratios =
["day_ratio","eve_ratio","night_ratio"]
df["time_ratio_std"] = df[ratios].std(axis=1)
```

파생피처 17개 생성 — jh_EDA_FINAL.ipynb

트러블슈팅 — 고객 이탈 예측

문제 상황을 가정한 설계로 신뢰성을 확보했습니다.

01

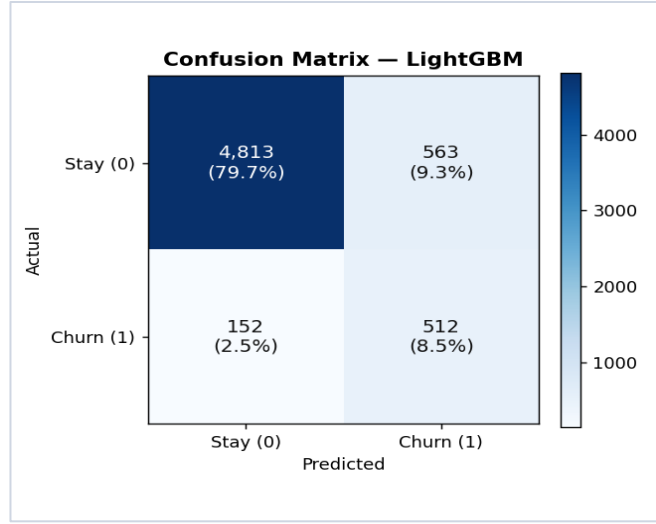
문제 이상치 단순 삭제 시 고위험 고객 정보 손실
해결 IQR로 파악 후 삭제 대신 파생피처로 흡수
효과 → **Recall 기여 · 정보 보존**

02

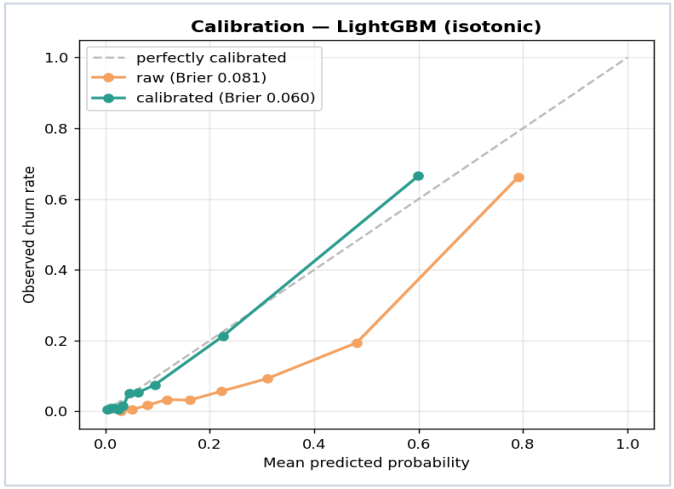
문제 클래스 불균형(이탈률 11%)으로 이탈을 놓침
해결 class_weight + 임계치 sweep
효과 → **Recall 중심 최적화(77%)**

03

문제 직관과 반대되는 패턴(저사용·단기 이탈 가정)
해결 가설 검증으로 장기·고사용이 최고위험 입증
효과 → **타깃팅 전략 재정립**



혼동행렬 — 이탈 탐지(Recall 77%)



확률 캘리브레이션 (Brier 0.060)

Thank You

읽어주셔서 감사합니다.

데이터 분석부터 모델, 서비스까지 — 끝까지 만들어내는 개발자가 되겠습니다.

Email twinking3594@gmail.com
GitHub github.com/han00jh
Kakao han9458